course number and updates the course grade--as simple as that. The code is for the addition of a new course (115 or 116) and replacement of grade for a course. The `revert` statement at the end is to handle any exceptional or invalid input.

The code for adding courses in other categories is similar. Here is the snippet for testing fulfillment of the core courses:

```
if(registeredStudents[msg.sender].prereq115 > 0 &&
        registeredStudents[msg.sender].prereq116 > 0) {

            preRequisitesSatisfied = true;
            emit preRequisiteSatisfied(personNumber);
            totalGPA += registeredStudents[msg.sender].prereq115 +
                        registeredStudents[msg.sender].prereq116;
```

In this case, the code checks to see whether courses 115 and 116 are present by checking whether the grade recorded is greater than 0. If so, it sets the flag `preRequisiteSatisfied` to `true`, emits an event indicating that fact, and adds up the grades to compute the overall certificate GPA. Also, note that the emitted event has the person number as a parameter, enabling external applications to access emitted events.

DESIGN CHOICES

There are choices in the data structures and approaches used in the design of the smart contract. Consider the choices and choose the optimal ones for the current design. Here are some design choices made for the DCC smart contract: